Mathematical Formulation of the Porous Navier-Stokes Solvers

Antigravity AI Project Authors

April 14, 2026

1 Introduction

This document outlines the formulation of the Orthogonal Sub-Grid Scales (OSGS) stabilization method and the nonlinear globalization strategy for the generalized porous Navier–Stokes equations. The algorithmic structure strictly separates the continuous finite element operator assembly from the discrete algebraic safeguarding mechanisms.

2 Solver Parameters

All bounds mapping to the internal runtime configuration are classified below.

2.1 Adaptive Discretization Tolerance Scaling

To rigorously prevent the iterative non-linear execution sequence from expensively over-solving coarse geometric boundaries, the fundamental terminal evaluation sequence tolerance (τ_{ftol}) is dynamically bound locally corresponding exactly to the spatial scaling bounds of the finite element function map $\mathcal{O}(h^{k_v+1})$. An exact evaluation tracks structurally: $\tau_{\text{ftol}} \leftarrow \max(\tau_{\text{ftol}}^{\text{base}}, \min(\tau_{\text{coarse_cap}}, \hat{\tau}_{\text{scale}} \times h^{k_v+1}))$, mapping safely over coarse matrices while natively locking densely at scale. The machine floor limit structurally shifts in identical bounds ($\tau_{\text{stagnation}} \leftarrow \tau_{\text{ftol}}/10$) preventing theoretical linesearch float saturation independently.

3 Algorithmic Abstractions and Code Equivalence

To bridge the gap between pure mathematical representations and the programmatic implementation, we rigorously define the equivalence between the algorithm functions presented in this section and their corresponding typed Julia structures.

Let Θ_{params} denote the grouped parameters for nonlinear iteration constraints, scalar tolerances, and bounds. When algorithms dynamically query these structures, they formally extract from:

$$\Theta_{\text{params}} = \{N_{\text{inner}}, N_{\text{inc}}, N_{\text{ls_max}}, \tau_{\text{ftol}}, \tau_{\text{xtol}}, \tau_{\text{stagnation}}, \alpha_{\min}, \gamma, c_1, F_{\text{div}}\} \tag{1}$$

4 Algorithmic Structure

Let the primary velocity-pressure state be $U_h = (\boldsymbol{u}_h, p_h) \in \mathcal{X}_h$. The continuous sub-grid projection variables are $\boldsymbol{\pi}_h = (\boldsymbol{\pi}_{h,u}, \pi_{h,p})^\top \in \mathcal{X}_h$.

To natively balance the numerical disparity between the momentum and mass conservation equations without *a priori* macroscopic scaling, we extract an element-wise scaling vector from the exact Jacobian at each iteration: $w_k = \max(|[\mathcal{J}]_{kk}|, \epsilon_{\text{mach}} \|\text{diag}(\mathcal{J})\|_\infty, \epsilon_{\text{mach}})$. The diagonally equilibrated dimensionless merit function is defined as:

$$\Phi_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2} \sum_{k=1}^{N} \left(\frac{b_k(\boldsymbol{x})}{w_k} \right)^2 \tag{2}$$

Table 1: Nonlinear and Staggered Solver Configuration Parameters

Symbol	**Code Property**	**Description**
N_{OSGS}	`stab_cfg.osgs_iterations`	Maximum number of outer staggered loops permitted.
N_{inner}	`stab_cfg.osgs_inner_newton_iters`	Iteration limit for each inner monolithic Newton-Raphson solve.
N_{inc}	`sol_cfg.max_increases`	Allowable successive iterations where the scalar merit function expands.
N_{plateau}	`mms_cfg.require_consecutive_passes`	Number of consecutive error verifications achieving steady reduction.
α_{min}	`sol_cfg.linesearch_alpha_min`	Minimum allowable geometric step size.
c_1	`sol_cfg.armijo_c1`	Armijo condition parameter evaluating sufficient residual descent.
F_{div}	`sol_cfg.divergence_merit_factor`	Expansion threshold evaluating catastrophic divergence bounds.
τ_{OSGS}	`stab_cfg.osgs_tolerance`	Tolerance evaluating acceptable outer state continuity.
τ_{proj}	`stab_cfg.osgs_projection_tol`	Tolerance evaluating sub-grid spatial projection limits.
τ_{ftol}	`sol_cfg.ftol`	Terminal nonlinear residual tolerance bounding the sequence ℓ^∞ norm.
τ_{xtol}	`sol_cfg.xtol`	Coordinate step norm limit halting sequence updates.
$\tau_{\text{stagnation}}$	`sol_cfg.stagnation_noise_floor`	Noise-floor limit preventing spurious machine-epsilon stalls.
τ_{error}	`mms_cfg.tau_err`	Plateau relative error bound verifying MMS limits.
τ_{warmup}	10^{-3} (Conceptual)	Relaxation floor bounding initial uncoupled OSGS inner iterations.
$\tau_{\text{coarse_cap}}$	10^{-4} (Conceptual)	Upper ceiling preventing over-solving on coarse topological meshes.
$\hat{\tau}_{\text{scale}}$	10^{-2} (Conceptual)	Scaling constant defining asymptotic grid error envelopes.
m_{warmup}	2 (Conceptual)	Transition frame halting OSGS decoupled initialization.
γ_{pert}	0.1 (Conceptual)	Homotopy constraint tracking exact exponential scalar limit 10^{-1}.

Table 2: Equivalence Between Mathematical Functions and Code Subroutines

Mathematical Function	**Code Signature Equivalent**
ExactNewtonPipeline($\boldsymbol{x}^0$)	`eval_algorithm_a_exact_newton!(x0, solver, op, ftol)`
SolveLinearSystem($\mathcal{J}_i, \boldsymbol{b}_i$)	`eval_linear_system_resolution!(dx, A, b, solver, cache)`
ArmijoLinesearch(...)	`eval_armijo_linesearch_pass!(x, x_old, b, dx, op, dir_deriv, solver, w, state)`
EvaluateSafeguards(...)	`eval_safeguard_termination_bounds!(i, solver, state)`
PicardFallback(...)	`eval_algorithm_b_picard_fallback!(x0, x0_backup, solver_picard, op_picard, ftol)`
OSGSFractionalRelaxation(...)	`eval_algorithm_c_osgs_staggered_fraction!(final_x0, X, Y, res_fn, jac_newton, stab_cfg, phys_cfg, dOmega)`
VerifyMMSPlateau(U_h^m, U_{ex})	`eval_mms_plateau_verification!(final_x0, op_newton_init, mms_cfg, mms, base_nls.ftol, dOmega)`
HomotopyExtrapolation(ϵ_{base})	`execute_outer_homotopy_perturbation_loop!(eps_pert_base, u_final, p_final, solver, mms_cfg)`

Algorithm 1: Algorithm A: ExactNewtonPipeline

Numerical Parameters: $\tau_{\text{ftol}}, N_{\text{inner}}, N_{\text{inc}}, \tau_{\text{stagnation}}, \tau_{\text{xtol}}$
Input: Initial geometric fields $\boldsymbol{x}^0 \in \mathcal{X}_h$
Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Initialize $\boldsymbol{b}_{\text{best}} = \|\boldsymbol{b}(\boldsymbol{x}^0)\|_\infty$, $I_{\text{up}} = 0$;
if $\|\boldsymbol{b}(\boldsymbol{x}^0)\|_\infty \leq \tau$ **then**
  return $\boldsymbol{x}^0$ (Exit: `initial_ftol`);
end
for $i = 1, \ldots, N_{inner}$ **do**
  Compute continuous variables: $\mathcal{J}_i, \boldsymbol{b}_i, \boldsymbol{w}, \Phi_{\text{curr}}, D = -2\Phi_{\text{curr}}$;
  $S_{\text{fail}}, \delta\boldsymbol{x} \leftarrow$ SolveLinearSystem$(\mathcal{J}_i, \boldsymbol{b}_i)$;
  if S_{fail} **then**
    Restore $\boldsymbol{x} \leftarrow \boldsymbol{x}_{\text{best}}$. **return** $(S_{\text{conv}} = \text{False}, I_{\text{iters}} = i, \|\boldsymbol{b}_{\text{best}}\|_\infty)$;
  end
  $S_{\text{step}}, \alpha, L_{\text{step}}, \hat{\boldsymbol{b}}, \hat{\Phi} \leftarrow$ ArmijoLinesearch$(\boldsymbol{x}, \boldsymbol{x}_{\text{old}}, \boldsymbol{b}, \delta\boldsymbol{x}, D, \Phi_{\text{curr}})$;
  if not S_{step} **or** $\|\hat{\boldsymbol{b}}\|_\infty < \boldsymbol{b}_{best}$ **then**
    $\boldsymbol{x}_{\text{best}} = \boldsymbol{x}, \boldsymbol{b}_{\text{best}} = \|\hat{\boldsymbol{b}}\|_\infty$;
  end
  $R_{\text{stop}}, S_{\text{conv}}, S_{\text{break}}, I_{\text{up}} \leftarrow$ EvaluateSafeguards$(i, S_{\text{step}}, L_{\text{step}}, \hat{\boldsymbol{b}}, \hat{\Phi}, \Phi_{\text{curr}})$;
  if S_{break} **then**
    if not S_{conv} **then**
      Restore $\boldsymbol{x} \leftarrow \boldsymbol{x}_{\text{best}}$.
    end
    return $(S_{\text{conv}}, i, \|\boldsymbol{b}_{\text{best}}\|_\infty)$;
  end
end

Algorithm 2: Algorithm A.1: SolveLinearSystem

Numerical Parameters: (None)
Input: Sparse continuous Jacobian $\mathcal{J}_i$, Residual array $\boldsymbol{b}_i$
Output: $(S_{\text{fail}} \in \{\text{True}, \text{False}\},\ \delta\boldsymbol{x} \in \mathbb{R}^N)$

---

try {
  Solve linear system functionally: $\mathcal{J}_i \delta\boldsymbol{x} = \boldsymbol{b}_i$;
} **catch** {
  return $(\text{True}, \delta\boldsymbol{x}_{\text{undefined}})$; `// Linear Solve Exception`
}
if $\delta\boldsymbol{x}$ *contains NaNs* **then**
  return $(\text{True}, \delta\boldsymbol{x}_{\text{undefined}})$; `// Catastrophic Math Failure`
end
return $(\text{False}, \delta\boldsymbol{x})$; `// Linear System bounded`

Algorithm 3: Algorithm A.2: ArmijoLinesearch

Numerical Parameters: $\alpha_{\min}, c_1, \gamma, N_{\text{ls_max}}$ (extracted from Θ_{params})
Input: State fields $\boldsymbol{x}, \boldsymbol{x}_{\text{old}}, \boldsymbol{b}$, step mapping $\delta\boldsymbol{x}$, exact directional derivative D, current merit Φ_{curr}
Output: $(S_{\text{accepted}} \in \{\text{True}, \text{False}\},\ \alpha \in \mathbb{R}^+,\ L_{\text{step}} = \|\alpha\delta\boldsymbol{x}\| \in \mathbb{R}^+,\ \|\hat{\boldsymbol{b}}\|_\infty \in \mathbb{R},\ \hat{\Phi}_{\boldsymbol{w}} \in \mathbb{R})$

Initialize $\alpha = 1.0, S_{\text{step_accepted}} = \text{false}$;
for $j = 1, \ldots, N_{ls_max}$ **do**
  $\boldsymbol{x}_{\text{trial}} = \boldsymbol{x}_{\text{old}} - \alpha\delta\boldsymbol{x}$;
  Compute $\hat{\boldsymbol{b}} = \boldsymbol{b}(\boldsymbol{x}_{\text{trial}})$ and $\hat{\Phi}_{\boldsymbol{w}} = \Phi_{\boldsymbol{w}}(\boldsymbol{x}_{\text{trial}})$;
  if not $isnan(\hat{\Phi}_{\boldsymbol{w}})$ **and** $\hat{\Phi}_{\boldsymbol{w}} \leq \Phi_{curr} + c_1\alpha D$ **then**
    $S_{\text{step_accepted}} = \text{true}$. **break**;
  end
  $\alpha = \gamma\alpha$;
  if $\alpha \leq \alpha_{min}$ **then**
    break;
  end
end
return $(S_{\text{step_accepted}},\ \alpha,\ \|\alpha\delta\boldsymbol{x}\|,\ \|\hat{\boldsymbol{b}}\|_\infty,\ \hat{\Phi}_{\boldsymbol{w}})$;

Algorithm 4: Algorithm A.3: EvaluateSafeguards

Numerical Parameters: $\tau_{\text{ftol}}, \tau_{\text{stagnation}}, \tau_{\text{xtol}}, N_{\text{inner}}, N_{\text{inc}}, F_{\text{div}}$ (extracted from Θ_{params})

Input: Iteration i, Validation $S_{\text{step_accepted}}$, step norm L_{step}, current residual $\|\hat{\boldsymbol{b}}\|_\infty$, scalar merit $\hat{\Phi}_{\boldsymbol{w}}$, base merit Φ_{curr}

Output: $(R_{\text{stop}},\ S_{\text{conv}} \in \{\text{True}, \text{False}\},\ S_{\text{break}} \in \{\text{True}, \text{False}\},\ I_{\text{up}} \in \mathbb{N})$

---

if not $S_{step_accepted}$ **then**
  return `"linesearch_failed"`, `false`, `true`, I_{up};
end
if $\|\hat{\boldsymbol{b}}\|_\infty \leq \tau$ **then**
  return `"ftol_reached"`, `true`, `true`, I_{up};
end
if $\|\hat{\boldsymbol{b}}\|_\infty \leq \tau_{stagnation}$ **then**
  return `"stagnation_noise_floor_reached"`, `true`, `true`, I_{up};
end
if $\hat{\Phi}_{\boldsymbol{w}} > \Phi_{curr} \cdot F_{div}$ **then**
  $I_{\text{up}} = I_{\text{up}} + 1$;
  if $I_{up} \geq N_{inc}$ **then**
    return `"merit_divergence_escaped"`, `false`, `true`, I_{up};
  end
end
else
  $I_{\text{up}} = 0$;
end
if $step_norm \leq \tau_{xtol}$ **then**
  return `"xtol_stagnation"`, `true`, `true`, I_{up};
end
if $i == N_{iters}$ **then**
  return `"max_iters_stagnation"`, `false`, `true`, I_{up};
end
return `""`, `false`, `false`, I_{up};

Algorithm 5: Algorithm B: PicardFallback

Numerical Parameters: $\tau_{\text{ftol}}, \tau_{\text{stagnation}}$

Input: Initial field guess $\boldsymbol{x}^0 \in \mathcal{X}_h$, Preconditioner backup $\boldsymbol{x}^0_{\text{backup}} \in \mathcal{X}_h$

Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Configure parameters zeroing sub-grid scales: $\boldsymbol{\pi}_h = \mathbf{0}$;
Attempt Newton resolution: $S_{\text{N}}, \cdots \leftarrow \text{ExactNewtonPipeline}(\boldsymbol{x}^0)$ targeting dynamic τ_{ftol};
if *S_N formally aborts (Exit via `"linesearch_failed"` or `"merit_divergence_escaped"`)* **then**
  Evaluate strictly guarded Picard Homotopy pass smoothing fields towards stable continuous initial geometry quadratic basins;
  Re-evaluate Newton sequence definitively mapping cleanly:
  $S_{\text{success}}, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty \leftarrow \text{ExactNewtonPipeline}(\boldsymbol{x}^0_{\text{smoothed}})$;
  if *Terminal evaluation fundamentally bypasses both `ftol_reached` and `stagnation_noise_floor_reached` structures* **then**
    return $(\text{False}, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty)$; `// Catastrophic Algebraic Sequence Stagnation`
  end
end
return $(\text{True}, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty)$; `// Mathematically locked globally bounded state`

Algorithm 6: Algorithm C: OSGSFractionalRelaxation

Numerical Parameters: $N_{\text{OSGS}}, m_{\text{warmup}}, \tau_{\text{warmup}}, \tau_{\text{ftol}}, \tau_{\text{OSGS}}, \tau_{\text{proj}}, \tau_{\text{stagnation}}$
Input: Converged steady state $U_h^{\text{ASGS}} \in \mathcal{X}_h$, Algebraic forms $(\mathcal{R}_u, \mathcal{R}_p)$
Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ \boldsymbol{\pi}_{h,u} \in \boldsymbol{V}_h,\ \pi_{h,p} \in Q_h,\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Pre-allocate strictly invariant orthogonal mappings: $M_u \leftarrow \int V \cdot U d\Omega, M_p \leftarrow \int Q \cdot P d\Omega$;
Initialize $U_h^0 = U_h^{\text{ASGS}}$;
Bootstrap initial sub-grid projections: $\boldsymbol{\pi}_{h,u}^0 = \Pi_{\boldsymbol{V}}(\mathcal{R}_u(U_h^0))$ and $\pi_{h,p}^0 = \Pi_Q(\mathcal{R}_p(U_h^0))$;
for $m = 1, \ldots, N_{OSGS}$ **do**
  Set inner tolerance limit: $\tau_{\text{inner}}^m = (m \leq m_{\text{warmup}})\ ?\ \max(\tau_{\text{ftol}}, \tau_{\text{warmup}}) : \tau_{\text{ftol}}$;
  Set $\boldsymbol{\pi}_h = \boldsymbol{\pi}_h^{m-1}$ fixed, resolve continuous algebraic system executing strict fraction mapping via ExactNewtonPipeline targeting limited iterations N_{inner};
  if *Monolithic sweep returns algebraic collapse natively* (`linesearch_failed` **or** `merit_divergence_escaped`) **then**
    Abort structural iteration immediately (mathematically divergent algebraic boundaries violated);
  end
  `/* Evaluate geometric state fraction shift explicitly evaluating explicit mass vectors linearly: */`
  $d_{\text{state}}^m = \max\left(\|\boldsymbol{u}_h^m - \boldsymbol{u}_h^{m-1}\|_{L^\infty(\Omega)},\ \|p_h^m - p_h^{m-1}\|_{L^\infty(\Omega)}\right)$;
  Update continuous evaluation geometries arrays natively: $\boldsymbol{\mathcal{R}}_u^m = \mathcal{R}_u(U_h^m)$ and $\mathcal{R}_p^m = \mathcal{R}_p(U_h^m)$;
  Update continuous orthogonal projections: $(\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m) = \left(\Pi_{\boldsymbol{V}}(\boldsymbol{\mathcal{R}}_u^m), \Pi_Q(\mathcal{R}_p^m)\right)$;
  `/* Independent Block-Weighted Anderson Acceleration */`
  (Optional) Filter limits through separated bounds using L^2 mass matrices:

$$\boldsymbol{\pi}_{h,u}^m \leftarrow \text{Anderson}(\text{History}_u, \boldsymbol{\pi}_{h,u}^{m-1}, \boldsymbol{\pi}_{h,u}^m, M_u)$$
$$\pi_{h,p}^m \leftarrow \text{Anderson}(\text{History}_p, \pi_{h,p}^{m-1}, \pi_{h,p}^m, M_p)$$

  `/* Define orthogonal projection drift explicitly evaluating algebraic block arrays: */`
  $d_\pi^m = \max\left(\sqrt{(\delta\boldsymbol{\pi}_{h,u}^m)^\top M_u \delta\boldsymbol{\pi}_{h,u}^m},\ \sqrt{(\delta\pi_{h,p}^m)^\top M_p \delta\pi_{h,p}^m}\right)$;
  if $(d_{state}^m \leq \max(\tau_{OSGS}, \tau_{stagnation}))$ **AND/OR** $(d_\pi^m \leq \max(\tau_{proj}, \tau_{stagnation}))$ **then**
    return `true`, $\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; `// Validated staggered boundary`
  end
end
return `false`, $\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; `// Catastrophic OSGS divergence abort`

Algorithm 7: Algorithm D: VerifyMMSPlateau

Numerical Parameters: $N_{\text{plateau}}, \tau_{\text{error}}$
Input: Candidate field state $U_h^m \in \mathcal{X}_h$, Exact analytical state $U_{\text{exact}} \in \mathcal{X}$
Output: $(S_{\text{plateau_valid}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Compute continuous tracking errors via explicit integrals: $E^m_{\boldsymbol{u},L^2} = \|\boldsymbol{u}_h^m - \boldsymbol{u}_{\text{ex}}\|_{L^2(\Omega)}$, $E^m_{p,L^2} = \|p_h^m - p_{\text{ex}}\|_{L^2(\Omega)}$;
Define local error variance tracking ratios over strictly isolated elements:

$$r_{\max} = \max\left(\frac{|E^m_{\boldsymbol{u},L^2} - E^{m-1}_{\boldsymbol{u},L^2}|}{\max(E^m_{\boldsymbol{u},L^2}, E^{m-1}_{\boldsymbol{u},L^2}, \epsilon_{\text{err}})},\quad \frac{|E^m_{p,L^2} - E^{m-1}_{p,L^2}|}{\max(E^m_{p,L^2}, E^{m-1}_{p,L^2}, \epsilon_{\text{err}})}\right) \tag{3}$$

;
if $r_{max} \leq \tau_{error}$ **then**
  $I_{\text{passes}} = I_{\text{passes}} + 1$;
end
else
  $I_{\text{passes}} = 0$;
end
if $I_{passes} \geq N_{plateau}$ **then**
  return `true`, $I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; `// mms_plateau_satisfied`
end
return `false`, $I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; `// mms_budget_exhausted`

Algorithm 8: Algorithm E: HomotopyExtrapolation

Numerical Parameters: $N_{\text{pert_max}}, \gamma_{\text{pert}}$
Input: Initial boundary limit ϵ_{base}
Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ \epsilon_{\text{converged}} \in \mathbb{R}_0^+,\ U_h^{\text{converged}} \in \mathcal{X}_h)$

for *attempt* $= 0, \ldots, N_{pert_max} + 1$ **do**
  $\epsilon_{\text{curr}} = \text{attempt} \leq N_{\text{pert_max}}\ ?\ \epsilon_{\text{base}} \cdot \gamma_{\text{pert}}^{\text{attempt}} : 0.0$;
  Inject synthetic bounded perturbation ϵ_{curr} into initial algebraic scalar boundary map;
  Execute strictly isolated mathematical assembly utilizing continuous bounded arrays (ExactNewtonPipeline, PicardFallback): $S_{\text{success}}, I_{\text{total}}, U_h^{\text{final}} \leftarrow \text{SolvePDEAlgorithm}(\epsilon_{\text{curr}})$;
  if *Assembly topologically returns* `success` == `true` **then**
    return `true`, $\epsilon_{\text{curr}}, \boldsymbol{x}_0$; `// Full sequence physically validated at homotopy level`
  end
end
return `false`, -1.0, nothing ; `// System physically collapsed completely across scalar spectrum`

5 Structural Orchestration and Convergence Analysis

The solver pipeline rigorously separates pure Newton-Raphson mathematics from physical boundary analysis through a cascading fallback strategy. By explicitly monitoring algorithmic exit bounds, the system aggressively tests fast evaluations while preserving mathematical robustness across dimensionless parameter extrema (e.g., $Re, Da \sim 10^6$).

5.1 Algorithmic Fallback Progression

Robustness is achieved by evaluating nonlinear failures geometrically rather than relying exclusively on continuous float exceptions.

1. **Aggressive Exact Newton Initialization (Algorithm A):** The solver unconditionally attempts to root the initial guess directly inside the quadratic convergence basin using pure analytic derivatives.

2. **Picard Globalization Fallback (Algorithm B):** If **Algorithm A** aborts aggressively due to linesearch depletion (`linesearch_failed`) or chaotic divergence (`merit_divergence_escaped`), the sequence retreats securely. It temporarily abandons exact Jacobian symmetry and invokes classical Picard iterations. Because the Picard Jacobian actively drops the advective coupling tensor, it stabilizes chaotic boundaries, smoothing the global fields strictly into the local quadratic basin before dynamically re-engaging Algorithm A seamlessly to securely polish the algebraic residual.

3. **External Homotopy Scaling (Top-Level Loop):** If the system is so stiff that both Exact Newton and guarded Picard bounds natively collapse completely failing to satisfy τ_{ftol} natively or the strict noise floor ($\tau_{\text{stagnation}}$), the orchestrator intercepts the failure and dynamically scales the initial boundary constraints. The pipeline physically dilutes the boundary injection forcing ($eps_pert = 1.0 \rightarrow 0.1\ldots$) forcing the continuous fields incrementally closer to the stable roots iteratively.

5.2 Dynamic Tolerance Resolution

A critical requirement for preserving computational efficiently is averting algebraic oversolve. Overdrilling a mesh geometrically scaled bounded at $\mathcal{O}(10^{-4})$ linearly down towards algebraic bounds $\tau_{\text{ftol}} = 10^{-10}$ wastes exponential linear solves globally against noise. The parameter bounds gracefully intersect using dynamically injected precision targets:

1. **Algebraic Spatial Limit:** The exact tolerance natively executed by **Algorithm A** is bounded logically utilizing $\tau_{\text{ftol}} \propto \mathcal{O}(h^{k_v+1})$, securely anchoring terminal roots two continuous orders of magnitude stricter than the continuous mesh limit.

2. **Stagnation Truncation:** The machine ceiling $\tau_{\text{stagnation}}$ intercepts the sequence exclusively intercepting asymptotic condition numbers geometrically before catastrophic UMFPACK pivoting bounds crash the arrays natively, returning `stagnation_noise_floor_reached` natively.

5.3 Fractional Verification Requirements

Instead of implicitly treating fractional evaluation success as mathematically rigorous, **Algorithm C (OSGS)** binds independent mathematical criteria: A fractional update sweep is deemed converged exclusively when *either* or *both* the spatial $L^\infty(\Omega)$ coordinate drift (d^m_{state}) *and/or* orthogonal projection vectors (d^m_π) independently hit continuous equilibrium τ_{OSGS}. If the inner linear fractional solver collapses (`linesearch_failed`), the staggered loop abruptly halts instantly recording a native fraction abort rather than falsely reporting stable limits conditionally.